Lab 3 — Architectural Justification

Self-Service Coffee Kiosk System | Keshav Agrawal (PES1UG24CS719)

Architecture Selection

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

The system is split into three layers: a Presentation layer (Touch Screen UI), a Business Logic layer (Order Manager, Payment Service, Receipt Service) and a Data & Hardware Access layer (Menu & Pricing Repository, plus adapters to the card gateway and the receipt printer). Each layer only calls the layer directly below it through a defined interface (IOrder, IPayment, IReceipt, IMenuData, ICardAuthorization, IPrinterDriver).

Reason 1 — The kiosk is a single, self-contained machine with a small, fixed scope

One kiosk serves one customer at a time, with only 3 coffee types, 2 sizes and one payment method. Microservices would add network hops, service discovery and container orchestration to a device that has no need to scale parts independently, and Client-Server would make the kiosk stop working whenever the central server or café network is down. A layered design runs entirely on the kiosk, is simple to build, test and deploy, and matches the clear Presentation → Business → Data flow of placing an order.

Reason 2 — Hardware and data details are isolated behind lower-layer interfaces

The touch screen, receipt printer and card processor are hardware/vendor specific. Because the Receipt Service talks only to IPrinterDriver and the Payment Service only to ICardAuthorization, the café can replace the printer model, switch card processor or change the menu/prices in the repository without touching the UI or the Order Manager. This separation of concerns makes the kiosk easy to maintain.

Security Advantage

Card data never reaches the Presentation layer. The UI only asks the Order Manager to “pay”; the Payment Service in the business layer is the single component that talks to the external gateway, over HTTPS/TLS, sending a tokenised card value and storing no card numbers locally (PCI-DSS). Because every request must pass through the layers in order, the UI cannot change prices or skip payment — the Order Manager recomputes the total from the Menu & Pricing Repository and only sends the order to the Receipt Service after IPayment returns an approved result.

Performance Benefit

All layers run in-process on the kiosk, so the UI → Order Manager → Repository calls are local method calls and a local database query rather than network requests. The small menu and price table can be cached in memory at start-up, so screens respond instantly to touch input and an order can be built and totalled in milliseconds. The only network call in the whole flow is the single card-authorisation request, which keeps checkout fast even on a busy café network.